Project Report

Data Storage Paradigms, IV1351

2025/11

Group 27 Project members:
[Sam Serbouti, serbouti@kth.se]
[Evelina Fridmane, fridmane@kth.se]

1 Declaration

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request. It is furthermore declared that the solution below is a contribution by the project members only, and specifically that no part of the solution has been copied from any other source (except for lecture slides at the course IV1351), no part of the solution has been provided by someone not listed as a project member above, and no part of the solution has been generated by a system.

2 Introduction

This report presents a conceptual model for the university staff management database provided to us by the course and details its subsequent conversion into logical-physical model developed using an ER diagram in Astah Professional. Finally, the report describes the implementation processes in PostgreSQL, covering database creation and data population. The university requires a system to facilitate course layout, planning and teaching load allocations - the two core tasks being "course layout and planning," which involves specifying hours needed for various teaching activities and "teaching load allocations" which assigns these activities to instructors.

The model is designed to satisfy the following requirements:

- Each course has a unique code, name, högskolepoäng (HP) and specified minimum and maximum number of students.

- A course instance represents a specific offering of a course in a given period (P1, P2, P3, or P4) within the academic year, including registration numbers.
- Teaching activities include lectures, labs, tutorials, seminars, examinations, administration and others, with flexibility to add more activities in the future.
- Preparation time for teaching activities is accounted for via multiplication factors specific to each activity type.
- Examination and administration hours are derived attributes calculated based on registered students and HP.
- Departments each have a manager who is also a teacher/employee.
- Teachers/employees are affiliated with departments, and their contact information, salary, and job titles are maintained to calculate teaching costs.
- Each teacher has a manager or supervisor.
- Teachers can be assigned to multiple course instances but cannot be allocated more than four different course instances simultaneously within a period.

3 Literature Study

3.1 Understanding the Conceptual Model

Our initial step involved studying chapters 1, 2 and 3 of the 7th edition of *Fundamentals of Database Systems* by Ramez Elmasri and Shamkant Navathe. These chapters provided a foundational understanding of database systems:

1. Chapter 1 covered the key properties of DBMSs, such as being self-describing, insulating programs from data, abstracting data, supporting multiple views and managing multiuser transactions, as well as highlighting advantages over file processing. It also introduced the basic steps in database design and outlined the roles of DBMS users and developers.

2. Chapter 2 introduced data models, the three-schema architecture, data languages and interfaces, the database system environment with its component modules and utilities, as well as centralized and multi-tier client/server architectures.

3. Chapter 3 emphasized the importance of high-level conceptual data models, detailing entities, attributes, relationships and weak entities in ER diagramming and provided a comparison between ER models and UML class diagrams.

This comprehensive overview allowed us to grasp the broader context of DBMSs and gain a general understanding of the stages following conceptual design, which was invaluable given this was our first database design project.

In addition, we drew technical insights from videos by Leif Lindbäck on conceptual and logical/physical modeling. His guidance, especially on common mistakes, helped establish boundaries while designing our logical/physical model using Astah Professional.

3.2 Building the Logical-Physical Model and SQL Scripts

To advance our work, we studied chapters 5 and 8 of *Fundamentals of Database Systems*, which focus on the relational data model, schemas, operations, constraints, relational algebra and relational calculus.

For practical guidance on writing SQL scripts to build the database, we referenced material from Database.guide.

This self-directed study was further supported by the course modules covering relational models, SQL, logical and physical modeling and normalization.

For examples of triggers in postgreSQL, we focused our attention to GeeksForGeeks's summary page. Finally, we took the official PostgreSQL documentation and GeeksForGeeks's explanation about UNION for reference concerning views.

4 Method

4.1 Designing the Logical-Physical Model

To design the university database, we employed the entity–relationship (ER) approach using IE notation, with diagrams constructed in the Astah Professional editor.

For building the logical-physical schema, we worked step by step in Astah Professional, referencing their guide on data types. Our process included the following steps:

- **Mapping Entities to Tables**: Each conceptual entity was translated into a separate table.
- **Adding Attributes for 0-1 or 1-1 Cardinality**: Attributes with cardinality 0..1 or 1..1 were included as columns. For attributes with cardinality 0..*, such as multiple phone numbers, these were not added as single columns.
- **Creating Tables for Multi-Valued Attributes**: Multi-valued attributes required the creation of additional tables.
- **Choosing Data Types**: For time slots, we used the `TIMESTAMP` type and `FLOAT` for floating-point numbers.
- **Defining Column Constraints**: Where appropriate, columns were marked as NOT NULL or UNIQUE[1].
- **Assigning Primary Keys to Strong Entities**: Primary keys were given to entities with their own identity - those not totally dependent on other entities. Entities recognized as strong included: person, employee, job_title, salary_history, department etc.
- **Added connections between Entities**: Added connections between entities according to conceptual model as well as task requirements (being mindful of cardinality, identifying relationship or not and direction).

[1]Column uniqueness was indicated as a note in `Astah`

4.2 SQL scripts for tables

Building database script starts with SQL lines relevant to users who do not use a GUI but directly work on a postgres console, so we introduce the commands to connect to the database. Then we have commands to drop old views, triggers, types and tables if they exist, which is needed to rerun the the script and create tables form the beginning each time. Then we wrote the script which creates two new types for our ENUMs. After that we populated the tables. The next step was to manually develop our SQL scripts step by step, creating each table individually. To prevent dependency issues (the department and the employee being dependent on each other), we first created tables without some foreign keys. In cases where cyclic dependencies were unavoidable, we temporarily commented out the affected foreign keys and added them later.

For data population, we used Perplexity to generate random entries, as it seemed to provide the fastest and most flexible way to produce data. This step ended up being long and tedious because of having to double check the mappings between primary and foreign keys between two tables. Since we used `INT GENERATED ALWAYS AS IDENTITY` it was postgres that created the primary IDs and we had to populate the tables one at a time, temporarily pausing the NOT NULL constraint, putting NULLs in the unknown foreign keys, then moving on to the next table etc.

4.3 SQL scripts for the trigger

The trigger is needed to prevent an employee from being involved with more than 4 courses per period per year. We thought of making a trigger when a user would want to add a planned activity to an employee and thus act on the `employee_activity` table. The trigger needs to first make a query and find the number of courses of the period and year linked to the planned activity the user wants to add. This is explained in the result section 5.2.1.

4.4 SQL scripts for the views

To meet the higher-grade requirements, we implemented the view, which helps us avoid using an application to calculate the total teachers hours. In the task explanation, we were given specific factors for teaching activities like Lecture, Seminar, etc, as well as two formulas for calculating Examination and Administration hours. View implements this logic directly in the database. It combines the manually entered hours (after multiplying them by their factor) with the automatically calculated Admin and Examination hours, presenting a complete list of the total course hours (for teachers) without needing any external calculations.

5 Result

All of our results can be found in our public github repository : *iv*1351_*project*

The ER conceptual model in Figure 1 represents the domain model of the university employee system.

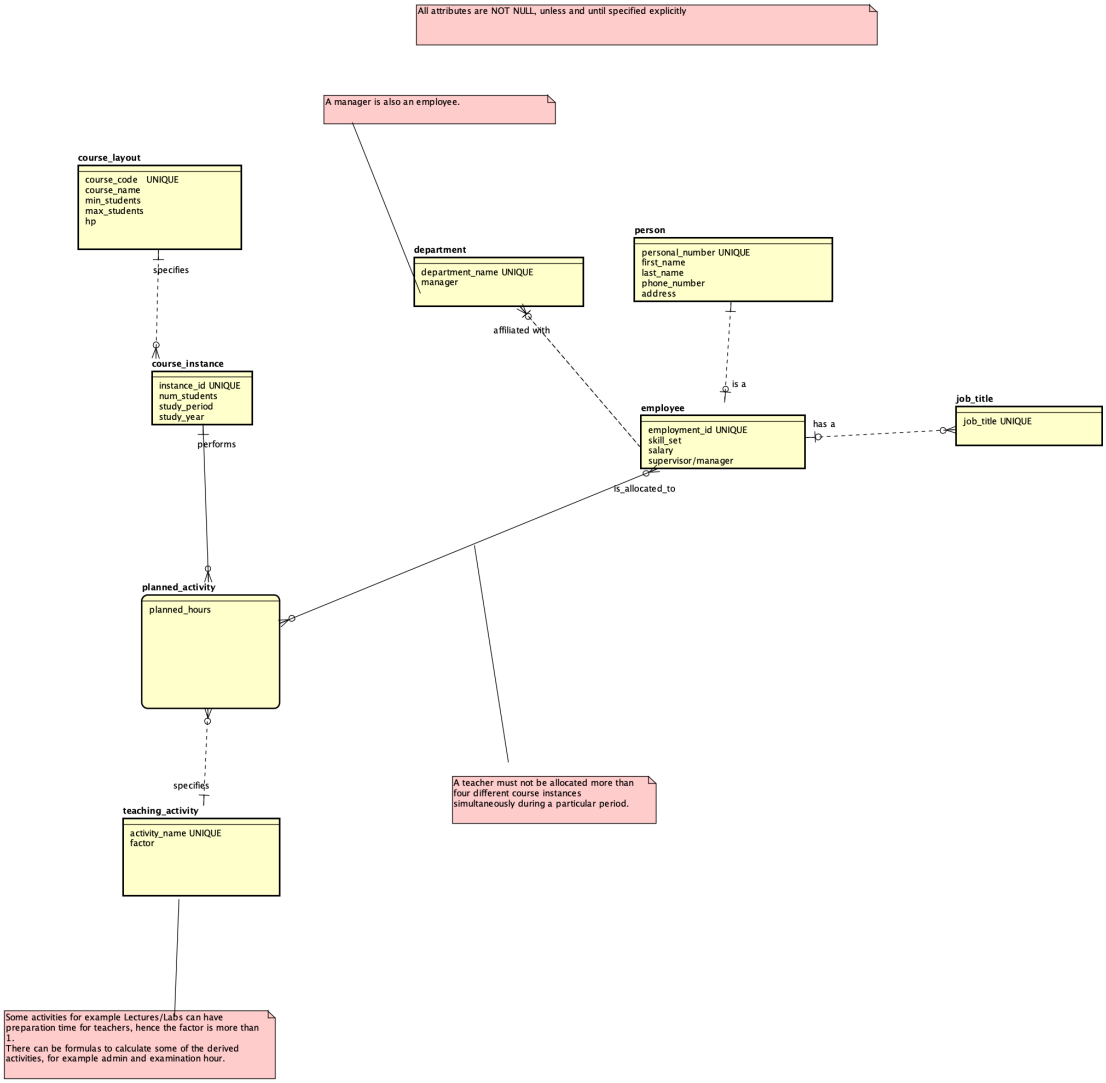

Figure 1: Screenshot of the conceptual model in Astah Professional

5.1 Logical and Physical model

To transform this conceptual model into a normalized logical-physical model, we followed the steps describes above and got set of normalized tables, including person, employee, course_layout, course_instance, and planned_activity. A more detailed description of the model and our implementation choices are presented in the section Discussion 6.

Contact and personal information management: we have a person as a parent entity, we assumed a person can have multiple phones and different people can have the same address (and a cross reference table ensures 3NF for this).

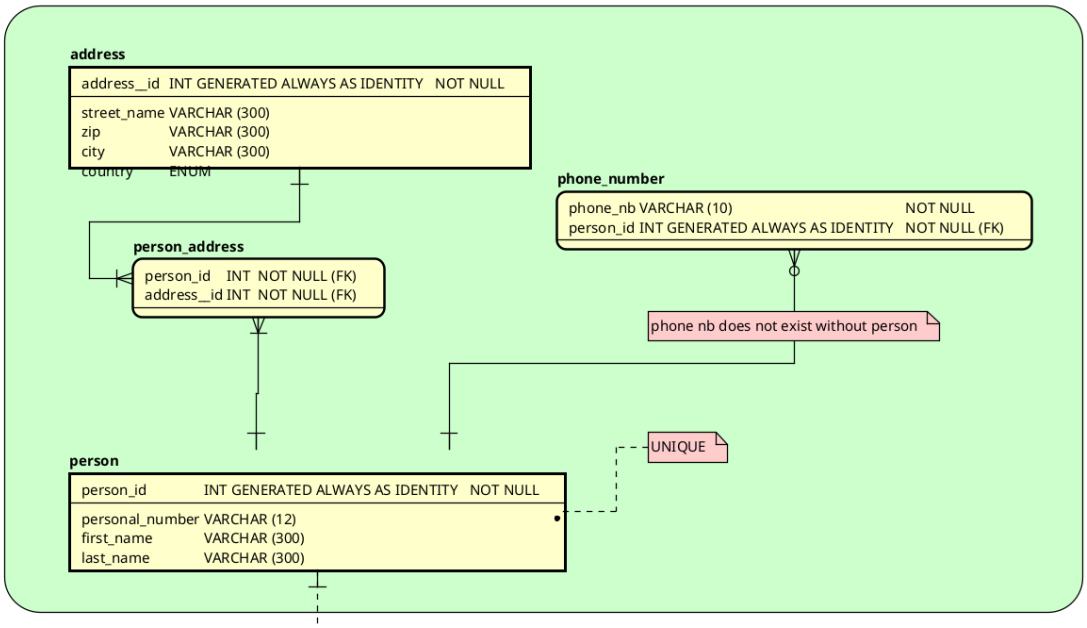

Figure 2: Screenshot of the person portion of the ER diagram in Astah Professional

Employee information: we used recursive loops between an employee and their manager (also an employee), an identifying relationship with skills (to avoid redundancies), a department table linked to employee through two relationships (one for affiliation and one for department management), we used a separate table for salary instead of a single column in employee in order to have its history, finally a cross reference table links an employee to their planned activities.

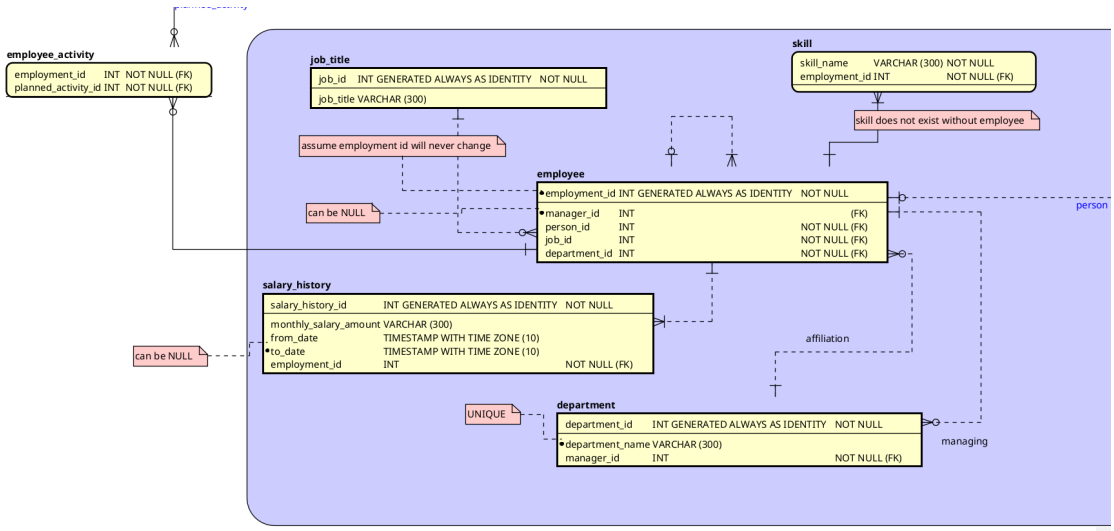

Figure 3: Screenshot of the employee portion of the ER diagram in Astah Professional

Course instances: all the entities are left the same as in the conceptual model, added attribute - layout_version

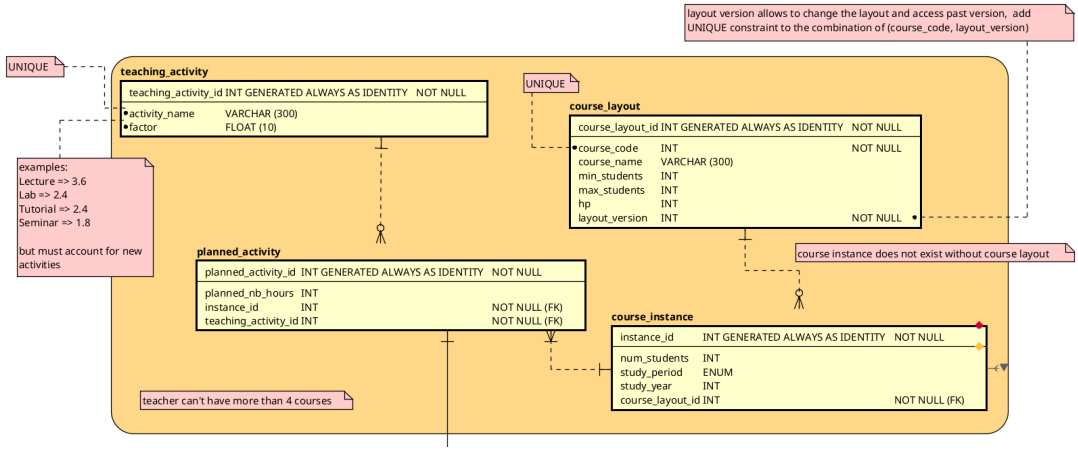

Figure 4: Screenshot of the course portion of the ER diagram in Astah Professional

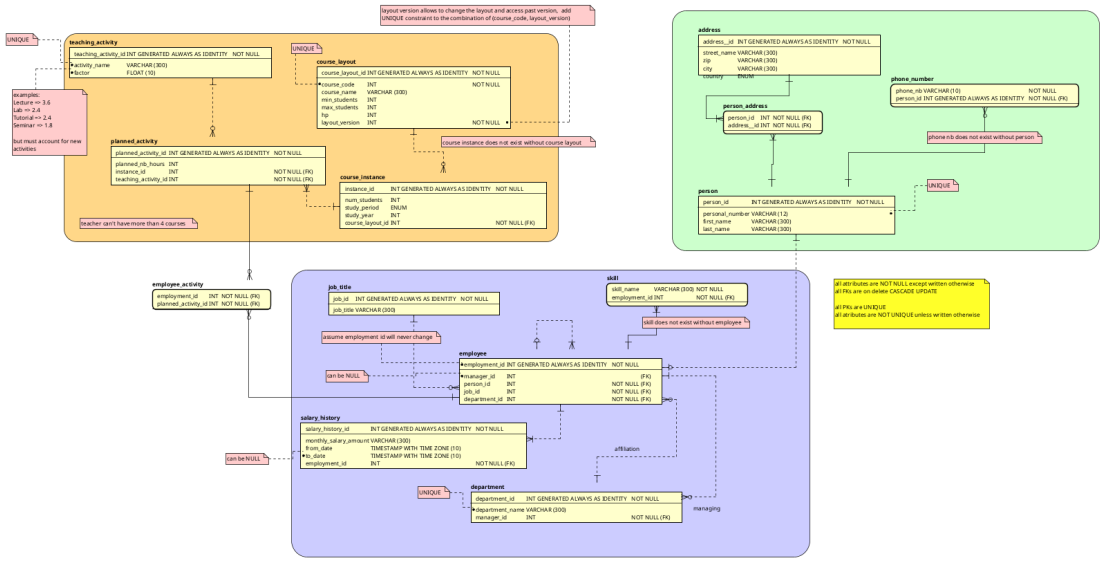

Figure 5: Screenshot of the ER diagram in Astah Professional complete

5.2 SQL scripts

The full view of our scripts is available on our public github. But here is a snippet of the employee build process:

```
...
CREATE TABLE employee (
        employment_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
        manager_id INT, --can be NULL if you have no manager
        person_id INT NOT NULL,
        job_id INT NOT NULL,
        department_id INT NOT NULL,
        --FOREIGN KEY (manager_id) REFERENCES employee(employment_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
        FOREIGN KEY (person_id) REFERENCES person(person_id)
        ON DELETE          RESTRICT ON UPDATE RESTRICT,
        FOREIGN KEY (job_id) REFERENCES job_title(job_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
        --FOREIGN KEY (department_id) REFERENCES department(department_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT
);
...
ALTER TABLE employee
        ADD CONSTRAINT fk_employee_manager
```

```
        FOREIGN KEY (manager_id) REFERENCES employee(employment_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT;
...
```

And here is a snippet of the employee populate process:

```
...
INSERT INTO employee (manager_id, person_id, job_id, department_id) VALUES
(NULL, 1, 25, 1), (1, 2, 30, 2), (2, 3, 24, 3), (3, 4, 21, 4), (4, 5, 22, 5),
(5, 6, 25, 6), (6, 7, 29, 7), (7, 8, 28, 8), (8, 9, 27, 9), (9, 10, 26, 10),
...
```

5.2.1 Trigger

The queries needed to obtain the number of courses in the period and year of the planned activity that the user wants to add to an employee can be divided into two parts :

1. First we need to obtain the course period, study year and course instance id of the new planned activity with an equijoin of the relevant tables:

$$J_1 = (\text{planned\_activity}) \bowtie \text{course\_instance})$$

$$S_1 = \sigma_{\text{planned\_activity\_id=NEW.planned\_activity\_id}}(J_1)$$

$$P_1 = \pi_{\text{study\_period, study\_year, instance\_id}}(S_1) \quad (1)$$

2. Then we need to count the courses corresponding to this course period and year:

$$J_2 = \text{employee\_activity} \bowtie \text{planned\_activity} \bowtie \text{course\_instance}$$

$$P_2 = \pi_{\text{employment\_id, instance\_id, study\_period, study\_year}}(J_2$$

$$Count = \gamma_{\text{employment\_id, study\_period, study\_year;count(instance\_id)}\rightarrow\text{course\_count...}}(P_2) \quad (2)$$

But to raise an exception only if the count of the courses is equal or above 4 **and** the course instance has not yet been assigned (otherwise we might count twice). This query must run before an insert in the employee_activity table.

6 Discussion

Overall, we took the decision of not propagating deletions via `CASCADE` for most of our columns and tables but instead use `ON RESTRICT` for updates and deletions : this will force users to update the database when deleting an employee tuple for example. Why? Because of how departments and employees are connected and that managers of employees are employees (ie recursive loops for this table) we felt on delete cascade was a dangerous approach (we wouldn't want to delete a whole chemistry department if the manager of that department is retired). Frequently updating the database certainly means more work for the user, but makes sure nothing is missing.

Moreover, we wanted to have a database as rigorous as possible (this was our own personal choice and we could have gone with a less restrictive approach) which is why almost all attributes are NOT NULL (unless when necessary). But this meant that also most attributes had to be NOT UNIQUE (unless for primary keys) and other exceptions like the personal number.

Following the tips of the course, all primary keys were implemented as `INT GENERATED ALWAYS AS IDENTITY`[2] and simply `INT` for foreign keys.

Our general structure for the model aims at avoiding a spider in the web (that induces high energie consumption and data overlaps) and instead use three parts : one for a person, one for an employee and one for a course (each referring to a real world concept).

6.1 Discussion on the person portion of the model

When assigning primary keys to strong entities, we decided not to use the personal number as the primary key. Instead, we introduced a surrogate key `person_id`, distinct from `personal_number`, to maintain a clear separation between real-world identifiers and system-generated ones. We considered the additional disk space required to be negligible. However, we had to ensure that the `personal_number` was UNIQUE (using the keyword in PostgreSQL) to avoid mistyping when users interact with the database.

For the `address` table, we decided to put additional columns (with zip, city, country) in the table itself and not into other tables. This might cause redundancy (especially if we consider that many of the employees live in the same city) and denormalization but for the tradeoff of easier maintenance and less overhead. We used an ENUM type for the country column of the address in order to simplify maintenance and avoid false differences[3]

We had initially thought of using cross referencing tables for both addresses and phone numbers (considering a person can have several phone numbers and/or addresses) for the sake of keeping the model in 3NF. But we decided to keep the cross reference table only for the address (two employees or more can share a house) but removed it for the

[2] in Postgres, this means an integer serial number picked by the database.

[3] For example : if someone types "UK" and another writes "United Kingdom", when the database manager whishes to know the list of countries of residence of its employees, it might count them as different. With our method, we avoid this possibility.

phone number since a phone number can only belong to one person. [4] But still we let it so that a person can have none or several personal phone numbers. Moreover, we decided to keep the phone number table linked to a person through an identifying relationship since a phone number cannot exist without its associated person.[5].

6.2 Discussion on the employee portion of the model

We had thought of using a **subtype** relationship between employee and person : an employee isA person (employee would then inherit all attributes) but decided against it since it was not encouraged in the scope of that course.

Contrary to how we separated a personal number (has real world significance) with a person_id (does not), we assumed that employment_id does not have a real world significance (ie does not play any role in the real world of the university), and will thus never change for an employee. This is why we picked it as sole primary key of an employee.

An employee can have a manager or not (otherwise we might get infinite university or cycles within the manager-employee logic) which is why the attribute that points to the employee tuple of the current employee's manager can be NULL.

Since an employee can have several skills and to avoid free text, we took the same approach for skills as for phone numbers. For better and quicker maintenance, the university could provide us with a list of skills of interest and we would use an ENUM type for the skill instead of a VARCHAR. This would avoid typos and possible redundancies.

An employee has only one job title, which can change. We could have gone for an attribute job title in the employee table but decided against when we worked on normalizing the model : to avoid redundancies (there can be many "teachers" or many "teacher assistants" for example), it was best to build a new table specifically for this.

To have an archived version of an employee's salary history that would not use traditional database archives with non-volatile memory, we took out the salary as an employee's attribute and put it in a new table with its own id, start date and end date (unless it's a current salary version) linked through a non-identifying relationships with an employee. This allows for a posteriori reviews and contemporary edits.

6.3 Discussion on the course portion of the model

We kept all the connections the same as in the conceptual model as they satisfied our needs. Since we identified a M:M connection between planned_activity and employee, we created a cross-reference table to connect them, ensuring the model remains normalized.

For the multiplication factor in the teaching activity table, we considered using BIT or ENUM types to restrict values. However, we decided against this and chose FLOAT instead. We did this to keep the database flexible because if more teaching activity

[4] We took the assumption that the university did not have any interest in the house phone number.

[5] Thus, if a row of the person table came to be deleted (if no longer relevant to keep in the database), its associated phone number row also will be deleted. This ensures no irrelevant data is kept in the database and is implemented by having a `person_id` FK in phone number as a PK.

options are added in the future with different factors, FLOAT can accommodate them without requiring schema changes. But for the study period, we had initially thought of using BIT but decided that it would be easier and more readable to use ENUM for defining the study periods.

To handle the fact that a course layout can change over time (for example, if the hp points change), we needed a way to store multiple versions of a layout. We solved this by creating a new, single primary key for the table called course_layout_id. This uses the `INT GENERATED ALWAYS AS IDENTITY` command. We then added a UNIQUE constraint to the combination of (course_code, layout_version) to enforce the rule that a specific version of a course can only exist once. This gives us a simple, single ID to reference in other tables.

To fulfill the requirement that a teacher can teach up to 4 courses, we left a note about that in the logical and physical model. We implemented a trigger function when building the database as it helps us to detect and throws an exception when another course teaching activity is trying to be added that would exceed this limit.

6.4 What we could have taken out

1. The ENUM for country in address and just put VARCHAR
2. The address cross-reference table. We decided to keep a cross-reference table for addresses to handle cases where employees live together. We could have taken this out and simply placed a foreign key to an address directly in the person table (or even put the address columns inside the person table).

6.5 What we could have added

1. Similarly to versions of a course layout, we could have added a `terminated` boolean (or BIT(1)) column to an employee for when their contract with the university is terminated. This would let us keep files of past employees (always good for posterity) and reuse the same data if the employee comes back again (by simply switching a bit). This would have a cost in memory space but avoids using too much non-volatile storage for archiving.
2. Per customer demand, we can create a predetermined list of skills of interests to the university.
3. We could have added a different column of address for house phone number, which we give the university the house phone number of an employee.
4. We could have added an **interest** table for an employee that would act with exactly the same logic as the current skill table. We decided to not implement it since it proved to be identical to skill. Per customer request, this would be a quick implementation.

6.6 Self assessment

Criteria	Notes	Discussion
Naming conventions followed? Names sufficiently explaining?	Yes, though the table salary_history could be renamed to salary_slip for preference.	The names are descriptive enough that you can understand the data without looking at documentation.
Crow foot notation correctly followed?	Yes, the diagram uses standard Crow's Foot notation to show cardinality.	We clearly distinguished between one-to-many relationships and the resolved many-to-many relationships (like employee_activity).
Model in 3NF?	Yes, all non-key attributes depend fully on the primary key and no transitive dependencies.	We separated repeating groups like skills and phone_numbers into their own tables.
All tables relevant? Table missing ?	Yes, every table handles a specific entity required	Nothing seems to be missing based on the project requirements.
Are there columns for all data that shall be stored ?	Yes	We checked the requirements to ensure fields like factor, hp, min_students and from_date are included.
Are all relevant column constraints and FK constraint specified ?	Yes, we used NOT NULL for mandatory fields and UNIQUE.	All foreign keys are set to ON DELETE RESTRICT.
Can all column types be motivated ?	Yes	Explained in details in the Discussion part.
Can the choice of PKs be motivated ? Are they unique ?	We used surrogate keys (GENERATED ALWAYS AS IDENTITY) for all tables to ensure stability.	Explained in details in the Discussion part.

Criteria	**Notes**	**Discussion**
Are all relations relevant ? Missing relation ? Correct cardinality ?	Yes, the many-to-many relationship between teachers and courses is correctly resolved	We also correctly modeled the recursive relationship for managers, where an employee references another employee.
Is it possible to perform all tasks listed in the project description ?	All requirements (see 2)	The examination and administration hours are computed through a view to allow 3NF
Are all business rules and constraints that are not visible in the diagram explained in plain text ?	We used anchor notes on our model for constraints.	If the customer wanted to make a distinction between a manager and a supervisor, we can change the logic of the employee portion of the model. But we decided to opt for simplicity.
Are there attributes which are calculated from other attributes and then written back to the database (derived attributes)? If so why?	We could have done so with the examinations and administration hours but opted out to keep 3NF and instead use views.	
Are tables (or ENUMS) always used instead of free text for constants such as P1, P2, P3, P4?	We used two custom types for our database : one ENUM for the study period and one for the name of a country.	If the customer prefers simplicity, we can take out the type for country name. If the customer wants a more restrictive system, we can build a bigger ENUM type for street names.

Table 1: Self-assessment based on the task's criteria

7 Comments About the Course

For this task, we attended all lectures given by Paris Carbone and dedicated around 20 hours to literature study.

Reading Chapter 2 of Fundamentals of Database Systems turned out to be both unnecessary and time-consuming for this assignment. In contrast, Leif Lindbäck's tutorials were very helpful in clarifying common mistakes to avoid. While working in Astah, we initially faced some confusion regarding foreign keys that act as primary keys in identifying relationships, overall understanding the difference between identifying and non-identifying relationships took some time.

The lecture on normalization proved particularly valuable for our work. Some additional preparation on SQL scripts-especially on views and triggers-could have been beneficial, but this also gave us a chance to explore and learn beyond the course material.

In total, we estimate having spent about 15 hours on this task.